

Diagrama UML SGDM (Versión para Imprimir)

```
classDiagram
class Usuario {
    <>
    #int id
    #int rolId
    #string nombreCompleto
    #string email
    #string contrasenaHash
    #bool estaActivo
    #DateTime fechaRegistro
    +obtenerPorId(id) Usuario
    +buscarPorEmail(email) array
    +crear(datos) int
    +actualizarUsuario(id, datos) bool
    +cambiarEstadoUsuario(id, estado) bool
    +eliminarUsuario(id) bool
    +obtenerTodosUsuariosConRoles() array
}
PerfilJugador --|> Usuario : "Subtipo Jugador"
PerfilOrganizador --|> Usuario : "Subtipo Organizador"
Usuario o-- Rol : "Asignado a Rol"
Usuario ..> Auditoria : "Registra en Auditoria"

class PerfilJugador {
    <>
    -int usuarioId
    -string apodoGamertag
    -string biografia
    -string nivelHabilidad
    -int puntosExperiencia
    -string pais
    +obtenerPerfil(usuarioId) array
    +actualizarGamertag(usuarioId, tag) bool
    +sumarExperiencia(usuarioId, xp) bool
}
PerfilJugador *-- EquipoMiembro : "Integra Equipos"

class PerfilOrganizador {
    <>
    -int usuarioId
    -string razonSocial
    -string sitioWeb
    -string telefono
    -string localidad
    -bool esVerificado
    +obtenerOrganizador(usuarioId) array
    +solicitarVerificacion(usuarioId) bool
}
```

```

PerfilOrganizador *-- Torneo : "Organiza Torneos"

class Rol {
    <>
    #int id
    #string nombreRol
    #int nivelPermiso
    +obtenerTodos() array
    +obtenerPermisos(rolId) array
}
Rol o-- Usuario : "Asigna Rol a Usuario"

class Torneo {
    <>
    #int id
    #int organizadorId
    #int juegoId
    #int modalidadId
    #string nombre
    #enum tipoFormato
    #enum estado
    #int maxParticipantes
    +crearTorneo(datos) int
    +cambiarEstado(id, estado) bool
    +contarTorneosActivos() int
    +obtenerUltimosTorneosCreados(limite) array
}
Torneo o-- PerfilOrganizador : "Organizado por"
Torneo o-- Juego : "Basado en Juego"
Torneo *-- Encuentro : "Contiene Encuentros"
Torneo *-- PosicionTorneo : "Tiene Tabla Posiciones"
Torneo *-- ParticipanteTorneo : "Tiene Participantes"
Torneo ..> ModuloLiga : "Usa Estrategia Liga"
Torneo ..> ModuloEliminacion : "Usa Estrategia Eliminacion"
Torneo ..> ModuloSuizo : "Usa Estrategia Suizo"

class ModuloLiga {
    <>
    -float puntosVictoria
    -float puntosEmpate
    -float puntosDerrota
    +generarFixtureTodosContraTodos(torneoId) bool
    +recalcularTablaPosiciones(torneoId) bool
}
ModuloLiga ..> Torneo : "Reglas de Torneo"
ModuloLiga ..> PosicionTorneo : "Actualiza Posiciones"

class ModuloEliminacion {
    <>
    -int totalRondas
    -bool tieneTercerPuesto
    +generarArbolLlaves(torneoId) bool
    +avanzarGanador(encuentroId, ganadorId) bool
}

```

```
ModuloEliminacion ..> Torneo : "Reglas de Torneo"
ModuloEliminacion ..> Encuentro : "Conecta Encuentros"

class ModuloSuizo {
    <>
    -int totalRondas
    -string criterioDesempate
    +generarRondaSuiza(torneoId, rondaNumero) bool
    +calcularBuchholz(torneoId, participanteId) float
}
ModuloSuizo ..> Torneo : "Reglas de Torneo"

class Encuentro {
    <>
    #int id
    #int torneoId
    #int rondaNumero
    #int participanteLocalId
    #int participanteVisitaId
    #int marcadorLocal
    #int marcadorVisita
    #int ganadorId
    #enum estado
    +registrarMarcador(id, local, visita) bool
    +validarResultado(id, arbitroId) bool
}
Encuentro *-- Torneo : "Pertenece a Torneo"
Encuentro o-- ParticipanteTorneo : "Enfrenta Participantes"

class PosicionTorneo {
    <>
    #int id
    #int torneoId
    #int participanteId
    #int partidosJugados
    #int victorias
    #int empates
    #int derrotas
    #int puntos
    #int diferenciaGoles
    +obtenerTablaOrdenada(torneoId) array
}
PosicionTorneo *-- Torneo : "Tabla del Torneo"

class ParticipanteTorneo {
    <>
    #int id
    #int torneoId
    #int usuarioId
    #int equipoId
    #enum estadoInscripcion
    #int semillaRank
    +inscribir(torneoId, userId, teamId) bool
    +confirmarCupo(id, confirmadoPor) bool
}
```

```

}
ParticipanteTorneo *-- Torneo : "Inscripto en Torneo"
ParticipanteTorneo o-- Usuario : "Jugador Individual"
ParticipanteTorneo o-- Equipo : "Equipo Grupal"

class Equipo {
    <>
    #int id
    #int capitanId
    #string nombre
    #string tag
    #string logoUrl
    +crearEquipo(datos) int
    +transferirCapitania(equipoId, nuevoCapitanId) bool
    +contarTotalEquipos() int
}
Equipo o-- PerfilJugador : "Capitan de Equipo"
Equipo *-- EquipoMiembro : "Tiene Miembros"

class EquipoMiembro {
    <>
    #int id
    #int equipoId
    #int jugadorId
    #enum rolEnEquipo
    +agregarMiembro(equipoId, jugadorId) bool
    +removerMiembro(equipoId, jugadorId) bool
}
EquipoMiembro *-- Equipo : "Roster de Equipo"

class Juego {
    <>
    #int id
    #string nombre
    #enum categoria
    +obtenerTodos() array
    +crearJuego(datos) bool
    +actualizarJuego(id, datos) bool
    +cambiarEstado(id) bool
}
Juego o-- Torneo : "Catalogo para Torneo"

class PoliticaContrasena {
    <>
    #int id
    #int longitudMinima
    #int expiracionDias
    #int historialCantidad
    +obtenerPoliticaVigente() array
    +guardarPolitica(datos, usuarioId) bool
    +validarPassword(password) array
}

class Auditoria {

```

```
<>
-int id
-int usuarioId
-string accion
-string descripcion
-string ipOrigen
-DateTime fechaHora
+registrar(accion, descripcion) void
+obtenerTodos() array
}
Auditoria o-- Usuario : "Registra Acciones"

class AdminControlador {
    <>
    +dashboard() void
    +obtenerUltimosJugadores(limite) array
    +obtenerUltimosOrganizadores(limite) array
}

class UsuarioControlador {
    <>
    +index() void
    +crear() void
    +guardar() void
    +editar(id) void
    +actualizarUsuario() void
    +cambiarEstado() void
    +eliminar() void
    +verComo() void
    +volverAdmin() void
}
UsuarioControlador ..> Usuario : "Gestiona Usuario"

class JuegoControlador {
    <>
    +index() void
    +crear() void
    +editar() void
    +actualizar() void
    +cambiarEstado() void
}
JuegoControlador ..> Juego : "Gestiona Juego"

class PoliticaContrasenaControlador {
    <>
    +index() void
    +guardar() void
}
PoliticaContrasenaControlador ..> PoliticaContrasena : "Gestiona Politica"

class AuditoriaControlador {
    <>
    +index() void
}
```

AuditoriaControlador ..> Auditoria : "Consulta Logs"

```
class Permiso {  
    <>  
    #int id  
    #string nombrePermiso  
    #string descripcion  
    +listarPermisos() array  
}  
Permiso o-- Rol : "Asociado a Rol"
```

```
class Modalidad {  
    <>  
    #int id  
    #string nombreModalidad  
    #int minJugadoresPorEquipo  
    +obtenerPorJuego(juegoId) array  
}  
Modalidad o-- Juego : "Pertenece a Juego"
```

```
class SolicitudEquipo {  
    <>  
    #int id  
    #int equipoId  
    #int jugadorId  
    #enum estado  
    +enviarInvitacion(eqId, jugId) bool  
    +responder(solId, aceptar) bool  
}  
SolicitudEquipo *-- Equipo : "Solicitud para Equipo"
```

```
class Conexion {  
    <>  
    -Conexion instancia  
    -PDO pdo  
    +getInstance() Conexion  
    +getBD() PDO  
}
```

```
class Sesion {  
    <>  
    +iniciarLogin(usuario) void  
    +validarAdmin() void  
    +esAdmin() bool  
    +destruirSesion() void  
}
```

```
class Validador {  
    <>  
    +requerido(valor) bool  
    +emailValido(email) bool  
    +enteroEnRango(v, min, max) bool  
    +sanitizar(cadena) string  
}
```

